Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 04

React: componente, JSX și props

Intrăm în frontend-ul template-ului: scriem o componentă reutilizabilă, o folosim pe o pagină nouă, „Despre proiect”, și o legăm de meniul aplicației.

Tema	Tema 7 din planul disciplinei (examene.pe.ub.ro): biblioteci și framework-uri JavaScript pentru interfață (React)
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` pornit (laboratorul 01), editorul și browserul
La final aveți	componenta `Card` și pagina `/despre` în aplicația React, salvate pe GitHub pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- explica ce este o componentă React și ce este JSX;
- transmite date unei componente prin **props**;
- crea o pagină nouă și o înregistra în rutele și în meniul aplicației;
- afișa o listă de componente cu `.map()` și atributul `key`.

2. Pe scurt: componente, JSX, props

React construiește interfața din **componente**: funcții JavaScript care întorc ce trebuie afișat [3]. O componentă se scrie o dată și se folosește oriunde, ca o etichetă HTML nouă: `<Card />`. Ce întoarce funcția se scrie în **JSX**, o sintaxă care seamănă cu HTML, dar este JavaScript. Diferențe de reținut: `className` în loc de `class`, `htmlFor` în loc de `for`, fiecare etichetă se închide (`<img />`), iar o expresie JavaScript se pune între acolade: `{titlu}`.

Props (proprietăți) sunt datele pe care o componentă le primește de la cea care o folosește, la fel cum o etichetă HTML primește atribute: `<Card title="Backend" />`. Componenta doar le citește, nu le modifică. Așa se separă **forma** (componenta) de **conținut** (props) [1], [4].

3. Pornirea

1. Porniți **Docker Desktop**. Deschideți folderul `daw-template-ti-info-2026-2027` în editor și terminalul integrat.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

3. Deschideți http://localhost:5173. Lăsați pagina deschisă: Vite o actualizează automat la fiecare salvare.

> **Sfat.** Dacă profesorul a anunțat noutăți pe `main`, preluați-le înainte de lucru: `git checkout main`, `git pull origin main`, `git checkout popescu.ion`, `git merge main --no-edit`, `git push origin popescu.ion`.

4. Să citim mai întâi codul existent

> **Unde.** Toate fișierele de azi sunt în `frontend/app/src/`. `App.jsx` conține meniul și rutele, iar folderul `pages/` conține câte un fișier pentru fiecare pagină.

Deschideți `pages/CategoriesPage.jsx` și căutați în el următoarele elemente. Le vom folosi ca model:

- linia `import` de la început și linia `export default function CategoriesPage()`: o pagină este tot o componentă;
- instrucțiunea `return ( ... )`, care întoarce JSX, cu atributul `className`;
- locul în care lista de categorii este afișată cu `.map(...)`, fiecare element având atributul `key`;
- liniile cu `useState` și `useEffect` (le explicăm în laboratorul 05).

Deschideți apoi `App.jsx` și identificați: liniile `import` ale paginilor, legăturile din meniu și liniile `<Route path="..." element={...} />`.

5. Componenta Card și pagina „Despre proiect”

5.1 Componenta reutilizabilă

În `frontend/app/src/` creați folderul `components`, iar în el fișierul `Card.jsx`:

```
export default function Card({ title, text, image, imageAlt = '' }) {
  return (
    <article className="info-card">
      {image && (
        <img className="info-card-image" src={image} alt={imageAlt} />
      )}
      <h2 className="info-card-title">{title}</h2>
      <p>{text}</p>
    </article>
  );
}
```

> **De ce.** Acoladele din parametri, `{ title, text, image, imageAlt }`, scot props-urile din obiectul primit. Expresia `{image && <img ... />}` afișează imaginea **doar dacă** a fost transmisă. `imageAlt = ''` este o valoare implicită. Prefixul `info-card` evită conflictele cu clasele CSS deja existente în template.

5.2 Pagina care folosește componenta

În `pages/` creați `AboutPage.jsx`:

```
import Card from '../components/Card';

export default function AboutPage() {
  return (
    <section>
      <h1>Despre proiect</h1>
      <p>Aplicație realizată la laboratorul de PW/DAW, 2026–2027.</p>
      <div className="info-card-grid">
        <Card title="Backend" text="Symfony 8 și PHP 8.5 expun un API JSON." />
        <Card title="Frontend" text="React 19 afișează datele primite de la API." />
        <Card title="Baza de date" text="PostgreSQL 18 păstrează datele." />
      </div>
    </section>
  );
}
```

5.3 Ruta și legătura din meniu

1. Deschideți `App.jsx`. Lângă celelalte importuri de pagini, adăugați:
   ```
   import AboutPage from './pages/AboutPage';
   ```
2. Lângă celelalte linii `<Route ...>`, adăugați:
   ```
   <Route path="/despre" element={<AboutPage />} />
   ```
3. În meniu, după modelul legăturilor existente, adăugați o legătură către `/despre`. Dacă legăturile folosesc `NavLink`, scrieți:
   ```
   <NavLink to="/despre">Despre</NavLink>
   ```
4. Dacă în meniu apare `Link` în loc de `NavLink`, folosiți `Link`; dacă legăturile au o clasă (`className`), copiați-o și pe cea nouă. Nu scrieți `<a href>`: ar reîncărca toată pagina.

5.4 Stilul

Adăugați la sfârșitul fișierului `App.css`:

```
.info-card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
  margin-top: 1rem;
}
.info-card { border: 1px solid #d1d5db; border-radius: 8px; padding: 1rem; }
.info-card-title { margin: 0.5rem 0; font-size: 1.2rem; }
.info-card-image { width: 100%; height: 140px; object-fit: cover; }
```

Rezultat. În meniu apare **Despre**. La click, adresa devine http://localhost:5173/despre, fără reîncărcarea paginii, și apar trei carduri așezate în grilă. Dacă pagina rămâne albă, deschideți consola (**F12**): cel mai des este o greșeală de import sau o etichetă neînchisă.

6. Liste: .map() și key

Când cardurile vin dintr-o listă de date (cum vin categoriile de la API), nu le scriem de mână, ci le generăm. Înlocuiți conținutul lui `AboutPage.jsx` cu:

```
import Card from '../components/Card';

const sections = [
  { id: 1, title: 'Backend', text: 'Symfony 8 și PHP 8.5 expun un API JSON.' },
  { id: 2, title: 'Frontend', text: 'React 19 afișează datele primite de la API.' },
  { id: 3, title: 'Baza de date', text: 'PostgreSQL 18 păstrează datele.' },
];

export default function AboutPage() {
  return (
    <section>
      <h1>Despre proiect</h1>
      <div className="info-card-grid">
        {sections.map((section) => (
          <Card
            key={section.id}
            title={section.title}
            text={section.text}
            image={section.image}
            imageAlt={section.imageAlt}
          />
        ))}
      </div>
    </section>
  );
}
```

De ce. `key` îi spune lui React care element este care, ca să actualizeze corect doar ce s-a schimbat. Folosiți un identificator unic și stabil (de exemplu `id`-ul din baza de date), nu poziția în listă. Fără `key`, consola afișează un avertisment.

7. Exerciții

1. Adăugați în lista `sections` un al patrulea element, „Docker”, cu o descriere scrisă de voi. Verificați că apare fără alte modificări.
2. Puneți o imagine în `frontend/app/public/` (de exemplu `echipa.jpg`) și adăugați-o unui element din `sections`: `image: '/echipa.jpg', imageAlt: '...'`. Cardurile fără imagine rămân neschimbate. Fișierele din `public/` se folosesc cu calea care începe cu `/`.
3. Creați componenta `components/PageTitle.jsx`, care primește props-urile `title` și `subtitle` și afișează un `<h1>` și un paragraf. Folosiți-o în `AboutPage`.
4. ★ (opțional) Dați componentei `Card` un prop `children` și afișați-l sub text, ca să puteți scrie `<Card title="..." text="..."><button>Detalii</button></Card>`.
5. ★ (opțional) Pe pagina „Despre proiect”, afișați numărul de secțiuni cu `{sections.length}`.

8. Salvarea lucrului și oprirea

```
git status
git add .
git commit -m "Laboratorul 04: componenta Card si pagina Despre"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Push. `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

> **Rezultat.** `git status` arată, înainte de commit, fișierele noi (`Card.jsx`, `AboutPage.jsx`) și cele modificate (`App.jsx`, `App.css`).

9. Ce arătați la final

- pagina http://localhost:5173/despre, deschisă din meniu, cu cardurile generate cu `.map()`;
- codul componentei `Card` și explicația, cu cuvintele voastre, a rolului props-urilor și al lui `key`;
- exercițiile 1–3 rezolvate și commit-ul vizibil pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – React:** https://www.w3schools.com/react/
- **W3Schools – React props:** https://www.w3schools.com/react/react_props.asp
- **React, în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PLZuysB_DjFARX5kWZGu8hvBbmw-3VWd8w
- **React pentru începători (Dave Gray, video în engleză):** https://www.youtube.com/watch?v=RVFAyFWO4go
- **React – prima componentă (documentație):** https://react.dev/learn/your-first-component
- **React – afișarea listelor:** https://react.dev/learn/rendering-lists

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[3] Meta Open Source, *React – Learn*. react.dev/learn

[4] Meta Open Source, *Passing Props to a Component*. react.dev

[5] Meta Open Source, *Writing Markup with JSX*. react.dev

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.